



CAPL



Introduction to the language

- CAPL is the abbreviation of Communication Access Programming Language
- It is exclusively used in Vector environment like CANalyzer, CANoe, vTESTstudio.
- It is mainly used to program network node modules
- It suits in application to:
 - Analyze specific messages or specific data
 - Analyze data traffic
 - Create and modify tool measurement environment
 - Design a custom module tester
 - Create a black box to simulate the rest of the network
 - Create a module simulator
 - Create a custom module manufacturing tester, diagnostic or service tool
 - Program a functional gateway between two different networks



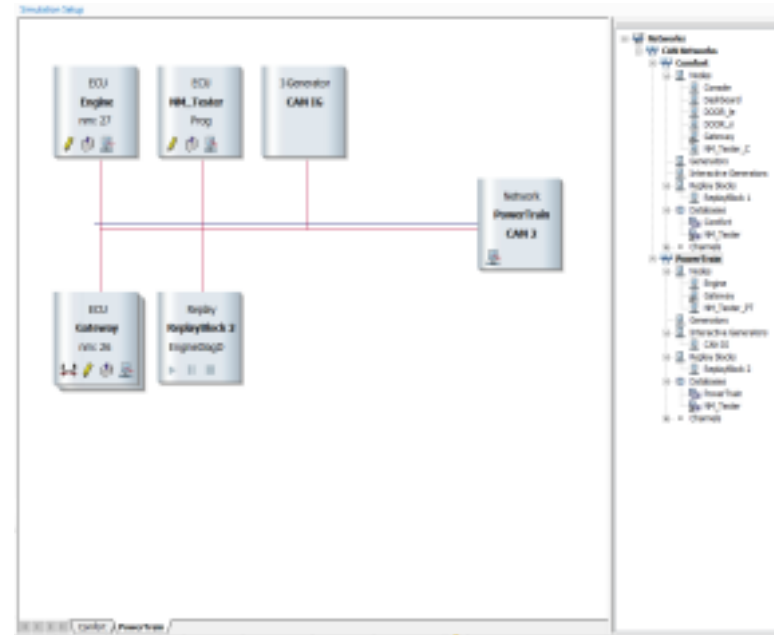


CANoe application

- CAPL is useful to write test for CANoe suite.
- This is an example of *Simulation Setup window* in CANoe. Each node in the figure is connected to a network (e.g. a CAN network).
- The depicted nodes are programmed with CAPL.
- On the right, there is the list of used *Networks*.
- One *Simulation Setup window* for each *Network*.
- Each network lists the programmed networks, (in Powetrain

There are Engine, NM_Tester_PT and Gateway).

- Each network lists the declared Interrupt Generator.
- Additionally, network can receive in input one or more Databases (DBC (CAN), LDF (LIN), XML (MOST),



FIBEX (FlexRay) and ARXML).



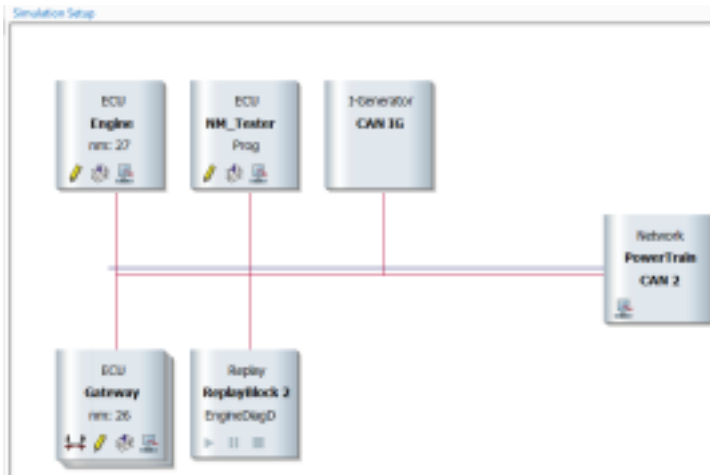
Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



CANoe application

- The pencil on the ECU allows you to *edit* and *open* the related CAPL program.
- The sheets on the ECU allows you to *compile* the related CAPL program.
- The compute on the ECU open Node panel window, which does not affect any CAPL program.

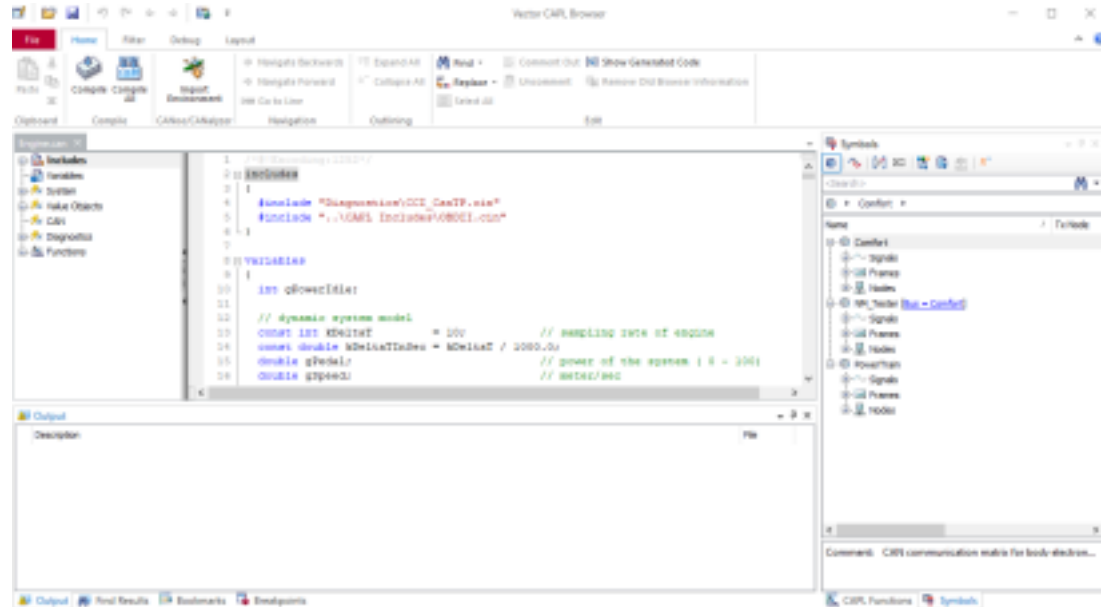
Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018





CANoe application

- Vector CAPL Browser is the editor to write CAPL programs.
- On the left, the list of
 - includes,
 - variables,
 - System (*on start, on timer xyz, etc.*),
 - Value Objects (*on signal change xyz, etc.*),
 - CAN (*on message*),
 - Diagnostics (*on diagRequest*)
 - Functions (CAPL functions from lib)
- On the right
 - CAPL Functions help
 - Symbols (database)
- A database is made of messages and environmental variables.





CAPL vs C

- CAPL is C based language
- It is event based, not interrupt driven
- C concept not supported in CAPL:
 - main(), header files, preprocessor, macro definition, file inclusion, conditional compilation, pointers, structure, enumeration, union, define, typedef, sizeof, external, register, etc.
 - Standard C library (CAPL has specific library called CAPL DLLs).
 - String constants (in CAPL you may use character array to overcome this).
 - Void (in CAPL is only used for function return type).
- In CAPL, you don't have to declare a function prototype before you can use it.
- CAPL offers alternative to some C functions:



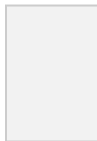
Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

Syntax and semantics

○ Comments. Examples:

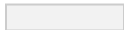


Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



Syntax and semantics

- Naming conventions for variables, array and functions. You can use letters or digits; but digits cannot be used in first character.





Syntax and semantics

- CAPL is case sensitive. So low and upper-case letters make the difference for user-defined variables. For example, CAPL treats these three declarations as different among each others.



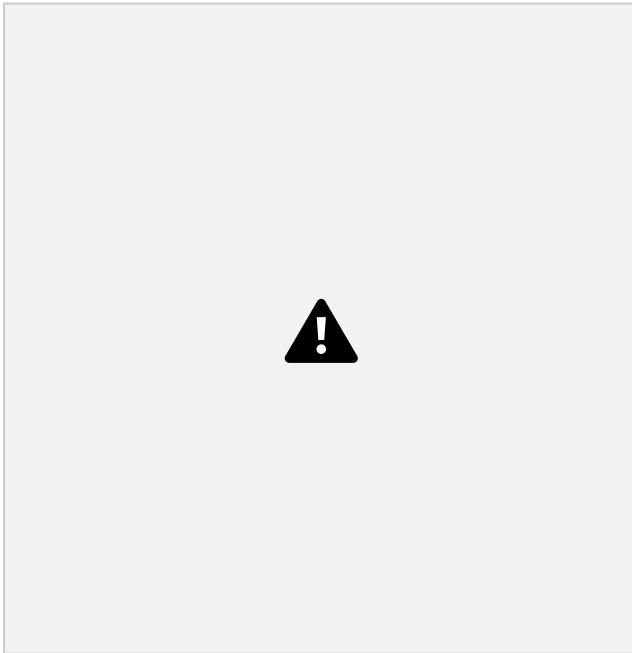
- Recommendations:
 - Because variables are case sensitive, you should have a consistent way of naming variables. It is recommended, that you adopt one of the three styles and then continue using that throughout your CAPL programs.
 - If it is expected that CAPL programs will be shared among a group of software programmers, it is also highly recommended that an internal CAPL coding standard be established.





Syntax and semantics

- Keywords cannot be used to name a variable or function. CAPL uses some of reserved keywords from the C programming language. Below is a list of reserved keywords, both supported and non-supported by CAPL.



Syntax and semantics

- The basic data types supported by CAPL are integer, character, and floating point. **Message**, **timer**, and **msTimer** are considered data types because they define a variable that symbolizes the kind of data they can store and operate. **Note:** Float signals defined in the database are 32 bits.



Syntax and semantics

- Declared global variables can be read or changed in any part of a CAPL program. They are declared in **Global Variables** window of the CAPL Browser.
- The simple integer data types are **dword**, **long**, **word**, **int**, **byte**, and **char**. While, **float** and **double** are synonyms and designate 64-bit floating-point numbers conforming to the IEEE standard. They are both compatible with each other and can be used in the same way in the C language.
- CAPL requires declarations for message and timers. Timers have to be globally declared. Messages can be declared globally or locally.
- Arithmetic is performed with 32-bit resolution for integers, and 80-bit resolution for floating point numbers.
- If local variables are used in event procedures, ensure they are defined first before any other code.
- Variables initialization is optional during their declarations. Below is a list of some

examples of declarations:

document are property of Kineton s.r.l. 2018



Strictly confidential. All contents in this

Syntax and semantics

- Below is a list of some examples of declarations:



- In contrast to standard C, the compiler initializes all numeric variables to **0** and all string-type variables to **null**.
- All message-type variables in CAPL are usually initialized to the transmit request state with its data field defaulted to **0**. The state represents the direction of transmission before the message is transmitted.

- Timer-type variables, on the other hand, are not automatically initialized and do not need to be initialized.

Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



Syntax and semantics

- In CAPL, unlike C, *local variables* are always declared statically. So, they are initialized only once – the first time the event procedure or user-defined function is executed. To avoid confusion, it might sometimes be necessary to declare all variables as global in the **Global Variables** window of the CAPL Browser.
- When variables enter the procedure, they assume the value they had at the end of the previous execution of the procedure. For example:



- **Note:** it may be helpful to consider that in a C program, **byte value = 10;** would be written **static byte value = 10;**
- The 1st time **myFunc** is called, it will print **value = 10** as expected. However, from the 2nd time it is called, it will print **value = 35** *as long as the measurement is running*.

Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

Syntax and semantics

- The preferred way to initialize non-static variables is to use a separate assignment statement after the variables have been declared. In the following example, the variables are reset to their initial values at the beginning of every function iteration:



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



Syntax and semantics

- CAPL supports type casting. Two ways are allowed, and they are performed like in C language. They are: automatic conversion, and the syntax (*data type*) *variable*.
- There are times when you will want to convert the value of a variable of one data type to another data type for purposes of evaluating an expression. For example:



- In the first case, 1.6 and 1.7 are added to yield 3.3; but the number is then truncated to the integer 3, an **int** variable.
- In the second statement, 1.6 is cast into an integer (1 in this example) before addition is applied to get value 1+1 or 2.

Syntax and semantics

- In CAPL, **array** are a collection of data items that all have the same name. Individual data items within an array must all be of the same type. The array name is always followed by square brackets that contain either the array size for a declaration statement, or an index value to specify the data location within the array. The first element is always the number 0, not 1. CAPL supports the following types of arrays:

- Integer arrays
 - Character arrays
 - One-dimensional arrays
 - Multi-dimensional arrays
- Character strings must include one additional character, the null character (`\0`), to indicate the end of the string. For example, the size of this array is 27 with index sequence from 0 through 26.
 - You can declare all, or partial element of an array by using `{}` brackets. For

example:



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

Syntax and semantics

- In CAPL, **matrix** can be of any dimensions. For example, a common matrix or two-dimensional array. It is analogous to what is used in math, such as the below 4 x 5 matrix. Definition and initialization of a two dimensional array M as shown in the example below:



- Note that except the last row, commas are required after each brace that closes off a row.
- To determine the number of elements in an array, use the **elCount()** function as shown in the below example:



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

Syntax and semantics

- To initialize a variable means to assign an initial or beginning value to it. In CAPL, this can be done in the same line as the variable declaration. When values are assigned during a declaration, they are considered constants. CAPL supports the following four types of constants:

- Integer constants
 - Floating point constants
 - Character constants
 - String constants
- CAPL supports integer constants in two different numbering systems -- decimal (base 10) and hexadecimal (base 16).



- In CAPL, a floating point constant is a base-10 decimal number. The constant must contain a decimal point or an exponent or both, as seen in the example below:



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

Syntax and semantics

- A character constant is a single character surrounded by apostrophes. CAPL supports the use of the ASCII character set.



- A string constant consists of a series of one or more consecutive characters enclosed by double quotation marks (“”).
- This is **NOT** used in CAPL. **#define TRUE 1 #define FALSE 0**

Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

Syntax and semantics

- A string constant consists of a series of one or more consecutive characters enclosed by double quotation

marks (“).



- Strings are stored in an array of data elements of type **char**. The last element contains the null character **\0**, which is used to indicate the end of the string.
- In CAPL, **enum** data types are treated in a manner similar to strings because values are initialized with symbolic names to represent integer constants. Even though the CAPL language does not support the **enum** data type, the database editor allows the **enum** type to be defined for all attribute objects. When CAPL needs to reference an enum-type attribute, use the **strncpy()** function. Below is an example of how to get the value of an enum-type message attribute that is defined for the receiver node's symbolic name:



- CAPL operators include:
 - Arithmetic Operators
 - Assignment Operators
 - Relational Operators
 - Boolean Operators
 - Bitwise Operators
 - Miscellaneous Operators
- CAPL does not support an exponential operator. ○ The table on the right for Arithmetic operators:



Syntax and semantics

- CAPL operators include:

- Arithmetic Operators
- Assignment Operators
- Relational Operators
- Boolean Operators
- Bitwise Operators
- Miscellaneous Operators

○ The table on the right for Assignment operators: Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



Syntax and semantics

- CAPL operators include:

- Arithmetic Operators
- Assignment Operators
- Relational Operators
- Boolean Operators
- Bitwise Operators
- Miscellaneous Operators

- The table on the right for Relational operators: Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

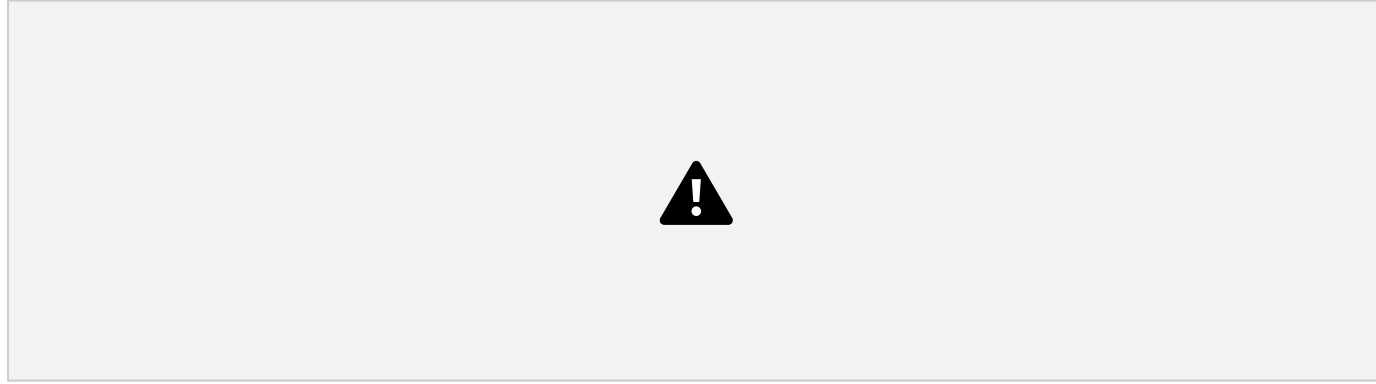


Syntax and semantics

- CAPL operators include:
 - Arithmetic Operators
 - Assignment Operators
 - Relational Operators
 - Boolean Operators

- Bitwise Operators
- Miscellaneous Operators

○ The table below for Boolean operators:



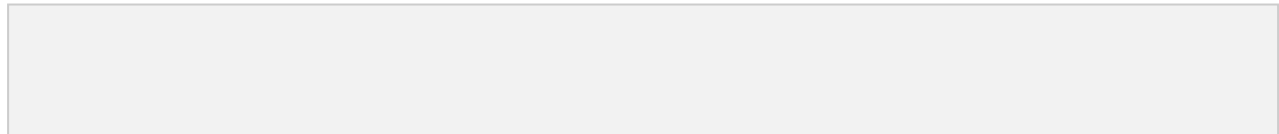
Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



Syntax and semantics

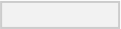
○ CAPL operators include:

- Arithmetic Operators
- Assignment Operators
- Relational Operators
- Boolean Operators



- Bitwise Operators
 - Miscellaneous Operators
- The table on the right for Bitwise op.

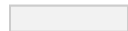
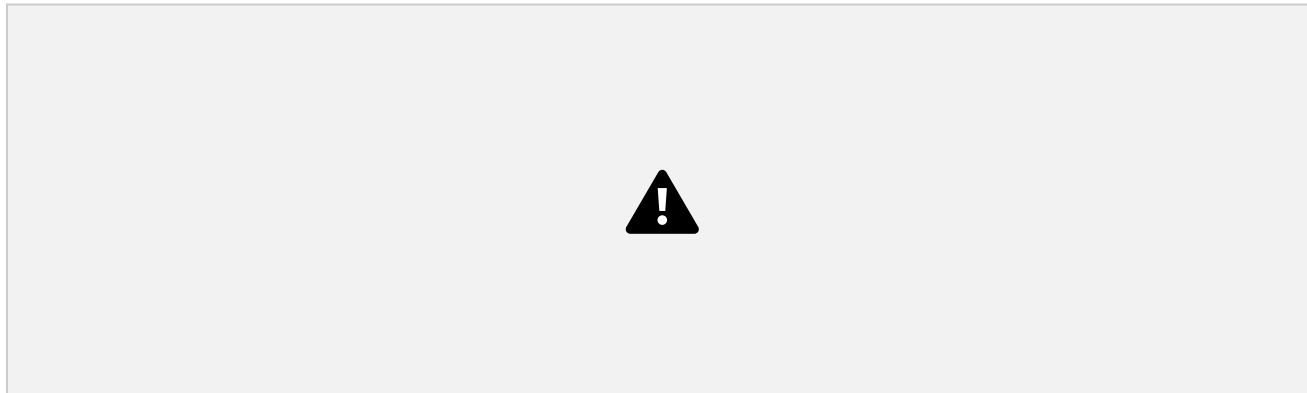
Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

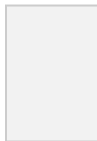




Syntax and semantics

- CAPL operators include:
 - Arithmetic Operators
 - Assignment Operators
 - Relational Operators
 - Boolean Operators
 - Bitwise Operators
 - Miscellaneous Operators
- The table below for Miscellaneous operators:

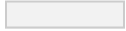




Syntax and semantics

- CAPL operators include:
 - Arithmetic Operators
 - Assignment Operators
 - Relational Operators
 - Boolean Operators
 - Bitwise Operators
 - Miscellaneous Operators
- The table with unsupported





Syntax and semantics

- In CAPL, several types of control statements are available including:

- Selective branching
- Looping
- Unconditional branching

- The **if-else** statement is used to evaluate an

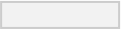
expression and then takes one of two possible actions, depending on whether the outcome of the evaluation is true or false. Generalized format:

```
if (expression) statement_1  
else statement_2
```

- Also in if- statement only

```
if (expression) statement;
```

Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018





Syntax and semantics

- In CAPL, several types of control statements are available including:
 - Selective branching
 - Looping
 - Unconditional branching
- The **switch** statement provides multiple control branching with a selection process that tests a value of an expression against a list of integer or character constants (**case** statements). When a match is found, the associated statement sequence specific to that case is executed until a break statement is encountered or until the end of the switch statement is reached.
- The **default** statement is executed if none of the cases match the value of the expression. If none of the cases match the value of the expression and the default statement is not implemented, then no action will be taken by the switch statement. Below is the general format of a **switch** statement:



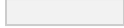


Syntax and semantics

- In CAPL, several types of control statements are available including:
 - Selective branching
 - Looping
 - Unconditional branching
- The **while** statement provides a looping mechanism during which a statement or group of statements is repetitively executed until some condition has been satisfied. The general form of the **while** statement contains both an expression and a statement. If the expression is true or evaluates to any nonzero value, then the statement is executed until the expression becomes false. Once the expression is evaluated as being false, then the loop terminates and the program resumes after the loop. The generalized format for the **while** statement is as follows:

while (expression) {statement};

- The expression is always evaluated before the statement is executed. Although this may eliminate performing a separate conditional test before executing a loop, remember that if the expression is false the first time through, then the statement is never executed.



Syntax and semantics

- In CAPL, several types of control statements are available including:
 - Selective branching
 - Looping
 - Unconditional branching
- The **do-while** statement repetitively executes a statement or group of statements until a condition has been satisfied. The general form of the **do-while** statement contains both a statement and an expression. If the expression is true or evaluates to any nonzero value, then the statement is executed until the expression becomes false. Once the expression is evaluated as being false, then execution resumes after the loop. The generalized format for the **do-while** statement is as follows:

do {statement}

while(expression);

Syntax and semantics

- In CAPL, several types of control statements are available including:
 - Selective branching
 - Looping
 - Unconditional branching
- The **for** statement also provides a looping mechanism. This statement includes an expression used for loop initialization, another expression that determines whether or not the loop is to continue, and a third expression that is executed after each iteration. The generalized format for the **for** statement is as follows:

```
for (expression_1; expression_2; expression_3)  
{statement}
```

Syntax and semantics

- In CAPL, several types of control statements are available including:
 - Selective branching
 - Looping
 - Unconditional branching
- The **break** statement is simply the word 'break' followed by the semicolon. Executing a **break** statement from within an enclosed **switch**, **do-while**, **while**, or **for** statement causes immediate termination to the loop. Example:
- A **continue** statement causes the program flow to go to the top of the enclosed **while**, **do-while**, or **for** loop, causing it to execute again. At the point that the **continue** statement is executed, any statements in

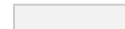
the loop that appear *after* the **continue** statement are automatically skipped. This is the opposite of the **break** statement.



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

Syntax and semantics

- In CAPL, several types of control statements are available including:
 - Selective branching
 - Looping
 - Unconditional branching
- A loop can also be exited by using a **return** statement with no argument. Most of the time, the **return** statement is used in a user-defined function to return a value or value of an expression to an event procedure.
- User-defined functions can return a value of any basic data type (for example, **int**, **float**, **long**, **double**, **char**, **byte**, or **word**).



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018



Syntax and semantics

- Because CAPL provides event-driven program design, the **this** keyword is used to reference the object that triggered the event.
- The **this** keyword is used to reference data from the object or the object value itself, such as a message or an environment variable (CANoe only) within the corresponding event procedure.
- The only event procedures that can use the **this** keyword are **on message**, **on envVar**, **on key**, **on errorframe** (only to get the CAN channel number), and the four CAN Controller events: **on busOff**, **on errorPassive**, **on errorActive**, and **on warningLimit**.



Strictly confidential. All contents in this document are property of

Kineton s.r.l. 2018



Events and event procedures

- A CAPL program is organized around event procedures. Each event procedure is associated with a single event. Whenever a predefined CAPL event occurs, the corresponding event procedure will execute.
- Events and their corresponding event procedures are grouped into functional classifications to simplify program organization. CAPL event procedures are functionally classified as follows:
 - Message events
 - Timer events
 - Keyboard events
 - Error frame events
 - CAN controller events
 - System (tool) events
 - Environment variable events (CANoe only)



Events and event procedures

- The keyword **this** is only used on a number of CAPL event procedures.
- The keyword **this** acts like a pointer, and essentially points at data related to the current event where the keyword is used. CAPL event procedures which use the keyword **this** and how they are used.



Kineton s.r.l. 2018

Strictly confidential. All contents in this document are

Events and event procedures

- When the same task applies to more than one message received from the bus or to different keystrokes on the keyboard. In CAPL, the * symbol is a wildcard that can be used to replace the key parameter for both the **on key** and **on message** event procedures.

- If the same action is performed for two or more keys, this symbol is used because it represents all key presses. All incoming messages with similar tasks may use this symbol as well.



- About priorities: with the addition of CAN channel-specific Message events, there are two rules CAPL follows to determine which event procedure to execute:
 - A specified CAN channel has precedence over the absence of a CAN channel.
 - A specified message ID number has precedence over a * symbol wildcard.

Confidential. All contents in this document are property of Kineton s.r.l. 2018

Events and event procedures

- Timers provide an easy way to trigger periodic events
- A timer is a programmable relative clock. After starting the clock with your pre-set amount of time, the timer

runs until it expires. The corresponding timer event procedure will be executed when the alarm expires. As a result, using timers is a three-step process:

- 1. Declare a timer variable
 - 2. Pre-set the timer in an event procedure (except the **preStart** event) or a user-defined function
 - 3. Define an **on timer** event procedure for that timer
- Based on typical timing requirements, two types of timers are available:
 - Millisecond timer - pre-set to a number in milliseconds
 - Seconds timer - pre-set to a number in seconds

Confidential. All contents in this document are property of Kineton s.r.l. 2018

Events and event procedures

- To use a timer, declare the timer by name in the **Global Variables** window of the CAPL Browser.

- A timer cannot be declared within an event procedure. If the unit of interest is in seconds, the timer is defined as **Timer**. If the unit of interest is in milliseconds, the timer is defined as **msTimer**.
- As an example, let's say we need to use 2 timers - one with a duration of a tenth of a second and the other for one minute. We could define these timers by using the following CAPL code:
 - **msTimer tenth_second_clock;**
 - **Timer one_minute_clock;**
- To start a timer, you must invoke the CAPL function **setTimer()**. We can start both of these timers by using the **setTimer()** function.
 - **setTimer(tenth_second_clock, 100);** // set timer to 100 milliseconds
 - **setTimer(one_minute_clock, 60);** // set timer to 1 minute

- When a timer or clock expires, the corresponding event procedure will be executed.
- To illustrate a delayed message transmission process, message 100 will be sent to the bus 20 milliseconds after the 'a' key is pressed as illustrated below:



Strictly confidential. All contents in this document are property of Kineton s.r.l. 2018

Events and

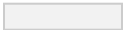
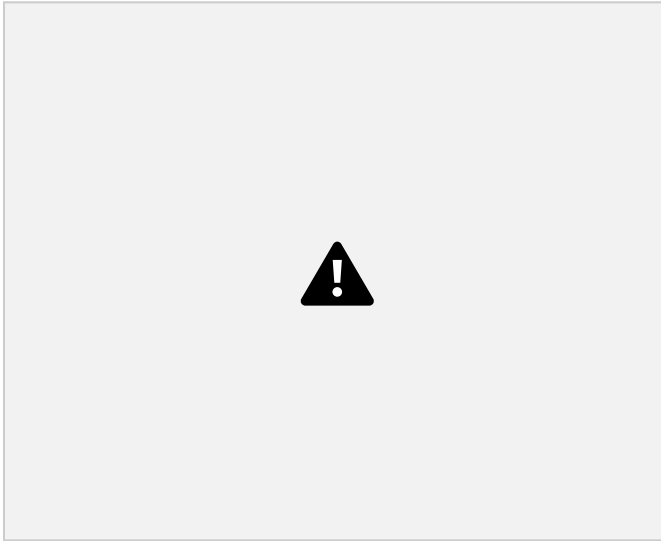
event procedures

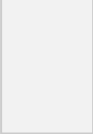
- When implementing a CAPL program to emulate node behaviours, you may have to implement a periodic timer. The most common way to create and use a periodic timer is shown in the following steps:
 - 1. Declare the timer in the **Variables** window of the CAPL Browser
 - 2. Set the timer in the **on start** event procedure
 - 3. Define an event procedure for the timer to perform a given task
 - 4. Reset the timer at the beginning or end of the task.
 - In the following example, a periodic message is sent using a timer.



event procedures

- A timer can be stopped by using the **cancelTimer()** function.
- The **cancelTimer()** function can indeed cancel a timer and avoid the errors that occur when trying to reset a timer that has not expired; however this function may also introduce a common mistake. In the example below, in addition to setting up a timer to send the periodic message every 100 milliseconds, an **on key** event is implemented to also send the message whenever the 'a' key is pressed.





Events and event procedures

- A timer can be stopped by using the **cancelTimer()** function.
- In this example, the user added an **on key** event to set a timer so as to output the message. The user knows that when the **on key** event is called, a timer error will occur because the timer cannot be set again while it is running. The user called the **cancelTimer()** function first, and then set the timer. This action, however, introduced another mistake.
- If the user presses the 'a' key faster than 100 milliseconds, the message will never be sent because the timer is cancelled every time before it expires. To send a message every 100 milliseconds and on the 'a' key, outputting the message within the **on key** event will correct the problem, as seen in the example below:

```
on key 'a'  
{  
output(msg1);  
}
```



Maria Pia Vaia
maria.vaia@kineton.it

Via E. Gianturco, 23
(presso complesso
Mecfond) 80146 Napoli

www.kineton.it